Assignment-3

Name:- Samiksha Sanjay Adake

Roll Number:- 2070120

CS-A

PRN:- 1251070524

Aim:

Develop a program to compute the fast transpose of a sparse matrix using its compact (triplet) representation efficiently

Objective:

- To identify and store a sparse matrix using compact (triplet) representation.
- To understand the concept of fast transpose for sparse matrices.
- To perform the transpose operation efficiently using row terms and starting positions.
- To reduce the time required for transpose compared to the simple transpose method.
- To understand efficient storage and processing techniques for sparse matrices in C++.

Theory:

A sparse matrix is a matrix in which most of the elements are zero. To save memory, only the non-zero elements are stored using compact (triplet) representation, where the first row contains the number of rows, columns, and non-zero elements, and each subsequent row stores the row index, column index, and value of a non-zero element.

The fast transpose algorithm is an improved version of the simple transpose. Instead of repeatedly scanning the sparse matrix for each column, it first counts the number of non-zero elements in each column and determines their starting positions in the transposed matrix. This allows every non-zero element to be placed directly in its correct position in a single traversal. As a result, the fast transpose method reduces the execution time and performs the transpose operation more efficiently.

Algorithm:

Algorithm 1: Compact Representation

1. Start.
2. Read the number of rows and columns.
3. Input all the elements of the matrix.
4. Count the non-zero elements.
5. Store the number of rows, columns, and non-zero elements in the first row of the triplet matrix.

6. Traverse the original matrix and store the row index, column index, and value of each non-zero element.
7. Display the compact representation.
8. Stop.

Algorithm 2: Fast Transpose

1. Start.
2. Read the compact (triplet) representation of the sparse matrix.
3. Store the number of rows, columns, and non-zero elements in the first row of the transposed matrix.
4. Count the number of non-zero elements present in each column.
5. Compute the starting position of each column in the transposed matrix.
6. Traverse each non-zero element of the original sparse matrix.
7. Place each element directly into its correct position in the transposed matrix using the starting positions.
8. Update the starting position after inserting each element.
9. Display the fast transposed matrix.
10. Stop.

Code:

```
#include <iostream>
using namespace std;

const int MAX = 100;

struct Term
{
    int row;
    int col;
    int value;
};

int rows, cols;
int matrix[MAX][MAX];
Term sparse[MAX], transpose[MAX];


void enterMatrix()
{
    cout << "Enter number of rows: ";
    cin >> rows;

    cout << "Enter number of columns: ";
    cin >> cols;

    cout << "\nEnter matrix elements:\n";

    for (int i = 0; i < rows; i++)
    {
        for (int j = 0; j < cols; j++)
        {
            cin >> matrix[i][j];
        }
    }

    int k = 1;

    sparse[0].row = rows;
    sparse[0].col = cols;
```

```
    sparse[0].value = 0;

    for (int i = 0; i < rows; i++)
    {
        for (int j = 0; j < cols; j++)
        {
            if (matrix[i][j] != 0)
            {
                sparse[k].row = i;
                sparse[k].col = j;
                sparse[k].value = matrix[i][j];
                k++;
                sparse[0].value++;
            }
        }
    }

    cout << "\nMatrix converted into Sparse Matrix successfully.\n";
}


void displaySparse()
{
    if (sparse[0].value == 0)
    {
        cout << "\nPlease enter the matrix first.\n";
        return;
    }

    cout << "\nSparse Matrix (3-Tuple Representation)\n";
    cout << "Row\tCol\tValue\n";

    for (int i = 0; i <= sparse[0].value; i++)
    {
        cout << sparse[i].row << "\t"
             << sparse[i].col << "\t"
             << sparse[i].value << endl;
    }
}
```

```
void fastTranspose()
{
    if (sparse[0].value == 0)
    {
        cout << "\nPlease enter the matrix first.\n";
        return;
    }

    int row_terms[MAX], starting_pos[MAX];

    int num_cols = sparse[0].col;
    int num_terms = sparse[0].value;

    transpose[0].row = num_cols;
    transpose[0].col = sparse[0].row;
    transpose[0].value = num_terms;

    for (int i = 0; i < num_cols; i++)
        row_terms[i] = 0;


    for (int i = 1; i <= num_terms; i++)
        row_terms[sparse[i].col]++;


    starting_pos[0] = 1;

    for (int i = 1; i < num_cols; i++)
        starting_pos[i] = starting_pos[i - 1] + row_terms[i - 1];


    for (int i = 1; i <= num_terms; i++)
    {
        int j = starting_pos[sparse[i].col];

        transpose[j].row = sparse[i].col;
        transpose[j].col = sparse[i].row;
        transpose[j].value = sparse[i].value;
```

```
        starting_pos[sparse[i].col]++;
    }

    cout << "\nFast Transpose\n";
    cout << "Row\tCol\tValue\n";

    for (int i = 0; i <= transpose[0].value; i++)
    {
        cout << transpose[i].row << "\t"
             << transpose[i].col << "\t"
             << transpose[i].value << endl;
    }
}

int main()
{
    int choice;

    do
    {
        cout << "\n========== MENU ==========\n";
        cout << "1. Enter Simple Matrix\n";
        cout << "2. Display Sparse Matrix\n";
        cout << "3. Fast Transpose\n";
        cout << "4. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice)
        {
        case 1:
            enterMatrix();
            break;

        case 2:
            displaySparse();
            break;

        case 3:
            fastTranspose();
```

```
            break;

        case 4:
            cout << "\nExiting Program...\n";
            break;

        default:
            cout << "\nInvalid Choice!\n";
        }

    } while (choice != 4);

    return 0;
}
```

Output:

PS C:\Users\LOQ\OneDrive\Desktop\SY Preparation\college notes\DS\Assignmentsss> & 'c:\Users\LOQ\.vscode\extensions\ms-vscode.cpptools-1.32.2-win32-x64\debugAdapters\bin\WindowsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-w0ktypbf.nhj' '--stdout=Microsoft-MIEngine-Out-qycnki1s.0az' '--stderr=Microsoft-MIEngine-Error-henfov0p.tkm' '--pid=Microsoft-MIEngine-Pid-fud2mfh5.os0' '--dbgExe=C:\mingw64\bin\gdb.exe' '--interpreter=mi'

========== MENU ==========
1. Enter Simple Matrix
2. Display Sparse Matrix
3. Fast Transpose
4. Exit
Enter your choice: 1
Enter number of rows: 3

Enter number of columns: 3

Enter matrix elements:
1 2 3 4 5 6 7 8 9

Matrix converted into Sparse Matrix successfully.

========== MENU ==========
1. Enter Simple Matrix
2. Display Sparse Matrix
3. Fast Transpose
4. Exit
Enter your choice: 2

Sparse Matrix (3-Tuple Representation)

Row	Col	Value
3	3	9
0	0	1
0	1	2
0	2	3
1	0	4
1	1	5
1	2	6
2	0	7
2	1	8
2	2	9

========== MENU ==========
1. Enter Simple Matrix
2. Display Sparse Matrix
3. Fast Transpose
4. Exit

```
Enter your choice: 3

Fast Transpose
Row     Col     Value
3       3       9
0       0       1
0       1       4
0       2       7
1       0       2
1       1       5
1       2       8
2       0       3
2       1       6
2       2       9

========== MENU ==========
1. Enter Simple Matrix
2. Display Sparse Matrix
3. Fast Transpose
4. Exit
Enter your choice: 4

Exiting Program...
PS C:\Users\LOQ\OneDrive\Desktop\SY Preparation\college
notes\DS\Assignmentsss>
```

Conclusion:

The program successfully computes the fast transpose of a sparse matrix using its compact (triplet) representation. It efficiently stores only the non-zero elements and performs the transpose operation using the row terms and starting positions arrays, reducing the execution time compared to the simple transpose method. This approach improves memory utilization and processing efficiency, helping in understanding optimized techniques for sparse matrix operations in C++.